MLSD Project Team 1 — Motor Insurance Claim Triage

SRN-PES1UG24AM321 ; NAME-Vipin V Lokesh

SRN-PES1UG24AM312 ; NAME-Samyuktha V

SRN-PES1UG24AM345 ; NAME-Ashvin R Lobo

SRN-PES1UG24AM330 ; NAME-Aariz Dudekula

Overview

The team built a multi-agent service that takes a motor-insurance First Notice of Loss (FNOL), validates it, works out the likely coverage position from the policy wording, and routes the claim with a cited justification. It is Mini Project 01 for ML System Design & AgentOps (UE24AM342AA5). The system is advisory only: it never approves or rejects a claim; a human handler decides.

The corpus is the Future Generali Two Wheeler Package Policy wording (v2.0), indexed into clauses tagged by section type (cover, exclusion, limitation, general) so the agents can check definitions, cover and exclusions together.

Architecture and tech stack

Six Docker Compose services keep intake, reasoning and telemetry isolated, so a crashed agent never takes down claim submission.

Service	Role
API (FastAPI + web UI)	Validates the FNOL against the data contract, quarantines rejects, returns 202 and enqueues
Redis	Job queue (RQ) and shared run state; keeps API and workers stateless
Agent workers	Run the LangGraph pipeline; scale with `--scale worker=N`
Prometheus + Grafana	Metrics and a claims dashboard

Service	Role
Jaeger	OpenTelemetry traces across API and workers

Each worker runs the agent graph in this order:

1. **Guard input** – injection and toxicity scan before any model call.
2. **Triage agent** – classifies own damage, third party, theft or total loss and extracts entities.
3. **Coverage agent** – tool loop over the policy corpus (search, clause lookup, FNOL fields).
4. **Verifier** – rejects any citation that does not resolve to retrieved text.
5. **Router + finalize** – picks the queue (fast-track, surveyor, investigation) and filters the output.

The LLM gateway has a circuit breaker and degrades gracefully; a stub LLM (1.5 s per call) is used for repeatable tests.

Guardrails, evaluation and testing

Guardrails are layered so the last layers do not trust the model. Prompt injection is scored with weighted rules (noisy-OR, threshold 0.6) on every string field that reaches a prompt, after Unicode normalisation. Severe threats escalate to a human; mild profanity is flagged but triage continues. An output filter blocks approval, rejection or payment-promise wording and escalates instead.

The repo has 50 test functions across six files (API, schemas, quarantine, guardrails, guardrail pipeline, cost report). Retrieval is evaluated against labelled scenarios in `eval/scenarios.json`. Two development incidents are logged with root cause and fix, e.g. malformed JSON causing a 500 instead of a 422 quarantine.

Load testing and results

The SLO was declared before testing: p95 end-to-end latency of 30 s or less and under 1% failed runs, with up to 6 claims in flight on 3 workers. The prediction (p95 ≈ ceil(N/W) × 10.5 s, breaking at 7 in flight) matched the Locust measurements within about 1 s.

Claims in flight (3 workers)	p95 (s)	Predicted p95 (s)	Throughput (runs/s)	SLO met
1	11.6	10.5	0.09	Yes

Claims in flight (3 workers)	p95 (s)	Predicted p95 (s)	Throughput (runs/s)	SLO met
6	22.2	21.0	0.27	Yes
7	31.7	31.5	0.28	No
18	65.0	63.0	0.29	No

Failures were 0% at every load level. CPU stayed under 3% on every service, confirming worker concurrency, not CPU, Redis or the API, is the bottleneck; one worker at 9 in flight gave 0.09 runs/s against 0.29 with three.

Conclusion

The service meets its declared SLO up to 6 concurrent claims on 3 workers with zero failed runs, and throughput scales linearly with workers. Correctness controls (verifier, layered guardrails, advisory-only output) sit inside the measured path rather than being traded for speed. Source: Vocod3r/MLSD_Project_Team1.